

Laboratoire 2 — Interruption, Timeur et PWM

Programmation sur STM32F103

Nom : Matisse Rhéaume-Viale

Cours : 243-356-LI - Programmation 3 - Microcontrôleurs et périphériques

Enseignant : Gabriel Fortin-Bélanger

Date : Lundi 21 Septembre

Groupe : 0001

0. Note sur l'Intelligence Artificielle

Le code, les explications, les résultats et les réponses aux questions n'ont pas été générés par l'IA. Si vous avez un doute sur une section, j'aimerais mieux que vous veniez me voir avant de tirer une conclusion.

Je pense qu'il est aussi important de rester prudent avant de conclure qu'un travail a été généré par l'IA simplement parce que son style « ressemble à de l'IA » ou parce qu'un détecteur l'indique. Ces outils ne permettent pas de déterminer avec certitude l'origine d'un texte et peuvent produire des faux positifs. Un doute devrait donc surtout servir à commencer une vérification, et non à constituer une preuve à lui seul. Dans ce genre de situation, je préfère qu'on me demande d'expliquer une section, de montrer mes fichiers de travail ou de démontrer ma compréhension du code avant qu'une conclusion soit tirée. Cette approche me semble aussi plus cohérente avec la PIEA, qui indique qu'un jugement d'évaluation doit être fondé, documenté, raisonné et interprété avec prudence.

Ce problème est d'ailleurs documenté. Une étude de Stanford a montré que plusieurs détecteurs avaient classé à tort **61,22 %** de textes rédigés par des étudiants non anglophones comme étant générés par une IA. OpenAI avait aussi développé son propre détecteur, mais l'a retiré en juillet 2023 en raison de son faible taux de précision; lors de ses tests, il ne détectait correctement qu'environ **26 %** des textes générés par IA et classait aussi certains textes humains comme étant générés par IA.

Je comprends donc qu'un professeur puisse avoir des doutes devant certains éléments d'un travail, mais je pense qu'un détecteur ou une impression sur le style devrait surtout servir à soulever une question, et non à constituer une preuve en soi.

L'IAG est encore relativement nouvelle et évolue extrêmement vite. Même pour quelqu'un qui travaille ou enseigne dans un domaine lié à l'informatique ou à la technologie, ou même pour un utilisateur régulier ou un *power user*, ça devient de plus en plus difficile de déterminer avec certitude ce qui a été fait ou non avec de l'IA.

Je comprends que certains éléments de mon document puissent paraître « trop

propres » ou faire penser à de l'IA, par exemple les tableaux bien structurés, la mise en page ou les encadrés de code. Par contre, je travaille directement en Markdown et je sais moi-même comment faire la mise en forme de mes blocs de code, par exemple avec ```.

Pour être transparent sur ma façon de travailler :

- **Mise en page et fichiers** : j'utilise parfois l'IA pour m'aider avec la mise en forme du Markdown afin d'obtenir un PDF propre, par exemple pour les sauts de ligne, l'indentation, le formatage du code ou certains problèmes de mise en page. Je peux aussi l'utiliser pour m'aider à organiser mes fichiers ou à nommer automatiquement mes captures d'écran.
- **Correction du français** : j'utilise **Antidote 12** pour corriger mes fautes et surtout pour améliorer certaines phrases quand elles sont difficiles à comprendre.

Tout le contenu technique, les démarches, les réflexions, le code et les réponses viennent de moi.

Si quelque chose vous semble douteux, je suis disponible pour vous montrer mes fichiers de travail, mon historique de modifications ou simplement vous expliquer en personne n'importe quelle partie du code ou de ma démarche.

1. Objectif du laboratoire

Faire clignoter la LED D1-1 sur PB6 selon 3 modes différents. Le bouton S1-3 sur PB5 sert à changer de mode.

- Mode 0 : LED allumée 500 ms et éteinte 500 ms avec SysTick.
- Mode 1 : PWM logiciel avec TIM2. Le duty cycle est réglé dans le code.
- Mode 2 : la LED change d'état à chaque interruption Update de TIM2.
- Chaque appui fait : Mode 0 → Mode 1 → Mode 2 → Mode 0.

Le projet est bare metal. Les registres sont utilisés directement, sans HAL.

Les fonctions créées dans ce laboratoire sont :

- SystemInit()
- main()
- LED_Init()
- LED_Set()
- LED_Toggle()
- Button_Init()
- Button_Read()
- SysTick_Init()
- SysTick_GetMs()
- SysTick_Handler()
- Timer2_Stop()
- Timer2_Start()

- Timer2_PWMInit()
- Timer2_InterruptInit()
- TIM2_IRQHandler()

2. Plan de test

2.1 Tableau des tests

Cas de test	Test	Étapes	Résultat attendu	Résultat obtenu
1	Mode 0 avec SysTick	Démarrer le programme et mesurer PB6.	500 ms ON / 500 ms OFF, donc une période de 1 s.	Période moyenne de 0,999 s, avec environ 500 ms ON / 500 ms OFF.
2	Mode 1 avec PWM logiciel	Appuyer une fois sur S1-3 et mesurer PB6.	PWM de 100 Hz. À 69 %, environ 6,9 ms ON / 3,1 ms OFF.	Fréquence moyenne de 100,11 Hz et environ 31 % à l'état haut sur PB6.
3	Mode 2 avec interruption TIM2	Appuyer une deuxième fois sur S1-3 et mesurer PB6.	Changement d'état chaque 500 ms, donc une période de 1 s.	Période moyenne de 0,999 s, avec un changement d'état environ chaque 500 ms.
4	Retour au mode 0	Appuyer une troisième fois sur S1-3.	Retour au clignotement 500 ms ON / 500 ms OFF.	Retour au mode 0 avec une période moyenne de 0,999 s.

2.2 Cas de test 1 — Mode 0 avec SysTick

Résultat attendu : 500 ms allumée et 500 ms éteinte.

Résultat mesuré : ΔT 3.9967278333333334 s Nfalling 4
Nrising 5

fmax 1.0009188434983316 Hz fmean 1.0008187114067104 Hz Tstd 0.00007835616540771647
s TposMean 0.4996544999999999

```

TposMin 0.4995924166666664
TposMax 0.4996994583333334
TnegMean 0.49952745833333345 TnegMin 0.4994895833333335
TnegMax 0.4995635833333334 Dutymean 0.500063572598151
Dutymin 0.5000514639105363
Dutymax 0.5000791704752382
fmean 1.0008187160229005
fmin 1.0007375018414613
fmax 1.0009188434983316
Tavg 0.9991819583333332 s

```

Capture de l'analyseur logique

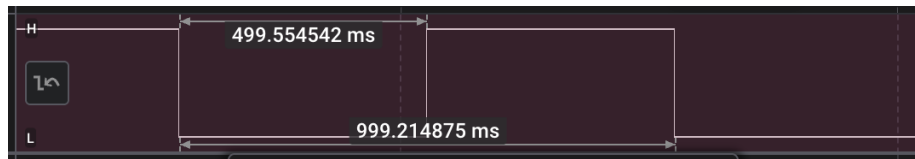


Figure 1: Mode 0 Saleae

Observation : la LED change d'état environ toutes les 500 ms. La période complète est proche de 1 s.

2.3 Cas de test 2 — PWM logiciel

Résultat attendu : PWM de 100 Hz. Avec PWM_DUTY_PERCENT à 69, mais puisque le Saleae affiche environ 31 % parce que la LED est active LOW, donc un duty cycle de 69 % dans le code veut dire que PB6 est à 0 pendant 69 % du temps et à 1 pendant environ 31 %.

Résultat mesuré : ΔT 0.18288966666666617 s Nfalling 19
Nrising 19
fmin 100.10427528676009 Hz fmax 100.12849823941522 Hz fmean 100.11492358937019
Hz Tstd 7.340696504253055e-7 s TposMean 0.0030964013157894635
TposMin 0.003096041666666679
TposMax 0.003096916666667312
TnegMean 0.0068921134259259005
TnegMin 0.006891083333332063
TnegMax 0.006893083333333795
Dutymean 0.30999659111567485 Dutymin 0.30994986276911324 Dutymax
0.3100352891121492
fmean 100.11492410006282
fmin 100.10427528676009
fmax 100.12849823941522
Tavg 0.00998852083333344 s

Capture de l'analyseur logique

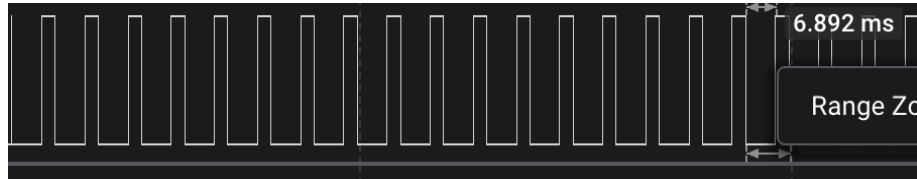


Figure 2: Mode 1 PWM Saleae

Observation : PWM avec un duty cycle sur Saleae de 31 %.

2.4 Cas de test 3 — Interruption TIM2

Résultat attendu : la LED change d'état toutes les 500 ms.

Résultat mesuré : ΔT 3.4956066249999997 s Nfalling 4

Nrising 4

fmin 1.0012351069406695 Hz fmax 1.0012444216605711 Hz fmean 1.0012409686500188

Hz Tstd 0.000005090557253580597 s TposMean 0.499403208333334

TposMin 0.499396666666666743 TposMax 0.499412625000000083 Tneg-

Mean 0.49934673611111063 TnegMin 0.499324916666666645 TnegMax

0.499361499999999907 Dutymean 0.500028271552949

Dutymin 0.5000232998620494

Dutymax 0.5000359209219057

fmean 1.0012516206208861

fmin 1.0012273796139843

fmax 1.0012800531078934

Tavg 0.9987605694444446 s

Capture de l'analyseur logique

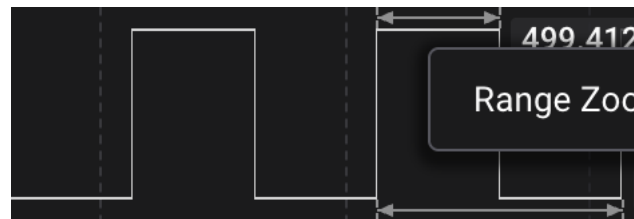


Figure 3: Mode 2 Saleae

Observation : la LED change d'état environ toutes les 500 ms selon l'interruption.

2.5 Cas de test 4 — Retour au mode 0

Résultat attendu : après le troisième appui, le programme revient au mode 0.

Résultat mesuré : ΔT 3.9967278333333334 s Nfalling 4
Nrising 5
fmax 1.0009188434983316 Hz fmean 1.0008187114067104 Hz Tstd 0.00007835616540771647
s TposMean 0.49965449999999999
TposMin 0.49959241666666664
TposMax 0.49969945833333334
TnegMean 0.49952745833333345 TnegMin 0.4994895833333335
TnegMax 0.49956358333333334 Dutymean 0.500063572598151
Dutymin 0.5000514639105363
Dutymax 0.5000791704752382
fmean 1.0008187160229005
fmin 1.0007375018414613
fmax 1.0009188434983316
Tavg 0.9991819583333332 s

Capture de l'analyseur logique

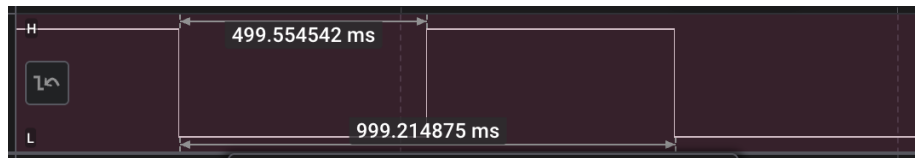


Figure 4: Retour au mode 0 Saleae

Observation : Retour au mode 0.

3. Questions d'analyse

Les sections suivantes utilisent le code actuel du projet.

3.1 Fonction SystemInit()

Capture du code de la fonction

```
void SystemInit(void)
{
    RCC_CR |= 1u; // active HSI

    while ((RCC_CR & (1u << 1u)) == 0u)
    {
```

```

    }

    RCC_CFGR &= ~3u; // prend HSI

    while ((RCC_CFGR & (3u << 2u)) != 0u)
    {
    }

    RCC_CFGR &= ~((0xFu << 4u) | (7u << 8u) | (7u << 11u)); // tout reste à 8 MHz
}

```

Explications du code et des configurations de registres utilisées

Cette fonction démarre le HSI et le choisit comme horloge système. Le HSI fonctionne à 8 MHz.

```
RCC_CR |= 1u;
```

Cette ligne met le bit HSION à 1. Elle active le HSI.

```
while ((RCC_CR & (1u << 1u)) == 0u)
```

La boucle attend que le bit HSIRDY passe à 1.

```
RCC_CFGR &= ~3u;
```

Cette ligne met les bits SW à 00. Le HSI est donc choisi comme horloge système.

```
RCC_CFGR &= ~((0xFu << 4u) | (7u << 8u) | (7u << 11u));
```

Les prescalers AHB, APB1 et APB2 restent à 1. Les bus restent donc à 8 MHz.

Registres utilisés

Registre	Adresse	Utilité
RCC_CR	0x40021000	Active le HSI et indique quand il est prêt.
RCC_CFGR	0x40021004	Choisit l'horloge système et les prescalers.

Justification avec la documentation officielle

Explication de la documentation :

On voit les bits HSION et HSIRDY dans RCC_CR. Dans RCC_CFGR, la valeur 00 pour SW choisit le HSI. Les prescalers à 0000 ou 000 ne divisent pas l'horloge.

8.3.1 Clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX83 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		PLL3 RDY	PLL3 ON	PLL2 RDY	PLL2 ON	PLL RDY	PLL ON	Reserved				CSSON	HSEBYP	HSE RDY	HSE ON
		r	rw	r	rw	r	rw					rw	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]				Res.	HSIRDY	HSION	
r	r	r	r	r	r	r	r	rw	rw	rw	rw		r	rw	

Figure 5: Register RCC_CR

8.3.2 Clock configuration register (RCC_CFGR)

Address offset: 0x04

Reset value: 0x0000 0000

Access: $0 \leq \text{wait state} \leq 2$, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during a clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				MCO[3:0]				Res.	OTGFS PRE	PLLMUL[3:0]				PLL XTPRE	PLL SRC
				rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC PRE[1:0]		PPRE2[2:0]		PPRE1[2:0]		HPRE[3:0]		SWS[1:0]		SW[1:0]					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r	rw	rw

Figure 6: Register RCC_CFGR

3.2 Fonction main()

Capture du code de la fonction

```
int main(void)
{
    unsigned char mode = MODE_0;
    unsigned char bouton;
    unsigned char ancienBouton;
    unsigned char boutonStable;

    unsigned int tempsBouton = 0u;
    unsigned int tempsLED = 0u;
    unsigned int temps;

    LED_Init();
    Button_Init();
    Timer2_Stop();
    SysTick_Init();

    bouton = Button_Read();
    ancienBouton = bouton;
    boutonStable = bouton;

    LED_Set(LED_ON); // commence allumé

    while (1)
    {
        temps = SysTick_GetMs();
        bouton = Button_Read();

        if (bouton != ancienBouton)
        {
            ancienBouton = bouton;
            tempsBouton = temps; // repart le debounce
        }

        if ((bouton != boutonStable) &&
            ((unsigned int)(temps - tempsBouton) >= DEBOUNCE))
        {
            boutonStable = bouton;

            if (boutonStable != 0u)
            {
                mode++; // prochain mode
            }
        }
    }
}
```

```

        if (mode > MODE_2)
        {
            mode = MODE_0;
        }

        Timer2_Stop();
        LED_Set(LED_OFF);

        if (mode == MODE_0)
        {
            LED_Set(LED_ON);
            tempsLED = temps;
        }
        else if (mode == MODE_1)
        {
            Timer2_PWMInit();
        }
        else
        {
            Timer2_InterruptInit();
        }
    }
}

if (mode == MODE_0)
{
    if ((unsigned int)(temps - tempsLED) >= TEMPS_LED)
    {
        tempsLED = temps;
        LED_Toggle(); // change au 500 ms
    }
}

__asm volatile ("dsb\n\twfi" ::: "memory"); // attend prochaine interruption
}

```

Explications du code et des configurations de registres utilisées

`main()` initialise la LED sur PB6, le bouton sur PB5, TIM2 et SysTick.

Le bouton doit rester stable pendant 20 ms avant que l'appui soit accepté. À chaque appui, `mode` augmente. Après le mode 2, il revient au mode 0.

- En mode 0, SysTick donne le temps en millisecondes. La LED change d'état après 500 ms.

- En mode 1, `Timer2_PWMInit()` démarre le PWM logiciel.
- En mode 2, `Timer2_InterruptInit()` fait une interruption TIM2 chaque 500 ms.

La soustraction `temps - tempsLED` permet de vérifier le temps passé sans bloquer le programme.

Registres utilisés

Aucun

Justification avec la documentation officielle

Explication de la documentation :

Les registres GPIO, SysTick et TIM2 utilisés par les trois modes sont expliqués dans les sections autre sections du rapport et puisque ce rapport n'est pas généré par l'IA, je n'ai pas le temps de remettre les explications.

3.3 Fonction LED_Init()

Capture du code de la fonction

```
void LED_Init(void)
{
    RCC_APB2ENR |= (1u << 3u); // active GPIOB

    LED_Set(LED_OFF); // commence éteinte

    GPIOB_CRL &= ~(0xFu << 24u); // efface PB6
    GPIOB_CRL |= (0x2u << 24u); // sortie push-pull 2 MHz
}
```

Explications du code et des configurations de registres utilisées

Cette fonction configure PB6 comme sortie pour la LED D1-1.

```
RCC_APB2ENR |= (1u << 3u);
```

Le bit IOPBEN active l'horloge du port GPIOB.

```
GPIOB_CRL &= ~(0xFu << 24u);
GPIOB_CRL |= (0x2u << 24u);
```

PB6 utilise les bits 27 à 24 de `GPIOB_CRL`. La valeur 0010 configure une sortie push-pull à 2 MHz. Les bits de PB5 ne sont pas changés.

Registres utilisés

Registre	Adresse	Utilité
RCC_APB2ENR	0x40021018	Active l'horloge de GPIOB.
GPIOB_CRL	0x40010C00	Configure PB6 en sortie push-pull à 2 MHz.

Justification avec la documentation officielle

8.3.7 APB2 peripheral clock enable register (RCC_APB2ENR)

Address: 0x18

Reset value: 0x0000 0000

Access: word, half-word and byte access

No wait states, except if the access occurs while an access to a peripheral in the APB2 domain is on going. In this case, wait states are inserted until the access to APB2 peripheral is finished.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART 1EN	Res.	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	Reserved		IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.	AFIO EN
	rw		rw	rw	rw	rw			rw	rw	rw	rw	rw		

Figure 7: Registre RCC_APB2ENR

9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 8: Registre GPIOx_CRL

Explication de la documentation :

On voit que les broches 0 à 7 sont configurées dans GPIOx_CRL. Pour PB6, MODE6 vaut 10 et CNF6 vaut 00.

3.4 Fonction LED_Set()

Capture du code de la fonction

```
void LED_Set(unsigned char etat)
{
    #if LED_ACTIVE_LOW == 1u
        if (etat == LED_ON)
        {
            GPIOB_BSRR = (LED_MASK << 16u); // PB6 à 0
        }
        else
        {
            GPIOB_BSRR = LED_MASK; // PB6 à 1
        }
    #else
        if (etat == LED_ON)
        {
            GPIOB_BSRR = LED_MASK; // PB6 à 1
        }
        else
        {
            GPIOB_BSRR = (LED_MASK << 16u); // PB6 à 0
        }
    #endif

    etatLED = etat;
}
```

Explications du code et des configurations de registres utilisées

Cette fonction allume ou éteint la LED. Dans le projet, LED_ACTIVE_LOW vaut 1. La LED est donc allumée quand PB6 est à 0.

GPIOB_BSRR = (LED_MASK << 16u);

Écrire dans la partie haute de BSRR remet PB6 à 0.

GPIOB_BSRR = LED_MASK;

Écrire dans la partie basse met PB6 à 1. etatLED garde aussi l'état demandé pour LED_Toggle().

Registres utilisés

Registre	Adresse	Utilité
GPIOB_BSRR	0x40010C10	Met PB6 à 1 ou à 0 sans changer les autres broches.

Justification avec la documentation officielle

9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x Reset bit y (y= 0 .. 15)

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Reset the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x Set bit y (y= 0 .. 15)

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Set the corresponding ODRx bit

Figure 9: Registre GPIOx_BSRR

Explication de la documentation :

La partie BS6 met PB6 à 1. La partie BR6, dans les bits 16 à 31 du même registre, remet PB6 à 0.

3.5 Fonction LED_Toggle()

Capture du code de la fonction

```
void LED_Toggle(void)
{
    if (etatLED == LED_OFF)
    {
        LED_Set(LED_ON);
    }
    else
```

```

    {
        LED_Set(LED_OFF);
    }
}

```

Explications du code et des configurations de registres utilisées

Cette fonction inverse l'état gardé dans `etatLED`. Elle appelle ensuite `LED_Set()` pour changer PB6.

Registres utilisés

Registre	Adresse	Utilité
Aucun directement	—	Le registre <code>GPIOB_BSRR</code> est utilisé dans <code>LED_Set()</code> .

Justification avec la documentation officielle

Explication de la documentation :

Cette fonction ne configure pas de registre. La commande de PB6 est expliquée dans la section `LED_Set()`.

3.6 Fonction `Button_Init()`

Capture du code de la fonction

```

void Button_Init(void)
{
    RCC_APB2ENR |= (1u << 3u); // active GPIOB

    GPIOB_CRL &= ~(0xFu << 20u); // efface PB5

    #if BUTTON_INTERNAL_PULL == 0u
        GPIOB_CRL |= (0x4u << 20u); // entrée flottante
    #else
        GPIOB_CRL |= (0x8u << 20u); // entrée pull-up/down

    #if BUTTON_INTERNAL_PULL == 1u
        GPIOB_BSRR = BUTTON_MASK; // pull-up
    #else
        GPIOB_BSRR = (BUTTON_MASK << 16u); // pull-down
    #endif
}

```

```
#endif
#endif
}
```

Explications du code et des configurations de registres utilisées

Cette fonction configure PB5 pour lire le bouton S1-3.

```
GPIOB_CRL &= ~(0xFu << 20u);
GPIOB_CRL |= (0x4u << 20u);
```

PB5 utilise les bits 23 à 20. Dans le projet, `BUTTON_INTERNAL_PULL` vaut 0. La valeur 0100 configure donc PB5 en entrée flottante.

Le reste du code permet aussi de choisir un pull-up ou un pull-down interne si la constante est changée.

Registres utilisés

Registre	Adresse	Utilité
RCC_APB2ENR	0x40021018	Active l'horloge de GPIOB.
GPIOB_CRL	0x40010C00	Configure PB5 en entrée.
GPIOB_BSRR	0x40010C10	Choisit le pull-up ou le pull-down seulement si cette option est activée.

Justification avec la documentation officielle

Explication de la documentation :

Dans `GPIOx_CRL`, `MODE5` à 00 donne une entrée. `CNF5` à 01 donne une entrée flottante. `CNF5` à 10 sert pour une entrée avec pull-up ou pull-down.

3.7 Fonction `Button_Read()`

Capture du code de la fonction

```
unsigned char Button_Read(void)
{
    unsigned char bouton;

    bouton = (GPIOB_IDR & BUTTON_MASK) ? 1u : 0u; // lit PB5
```

8.3.7 APB2 peripheral clock enable register (RCC_APB2ENR)

Address: 0x18

Reset value: 0x0000 0000

Access: word, half-word and byte access

No wait states, except if the access occurs while an access to a peripheral in the APB2 domain is on going. In this case, wait states are inserted until the access to APB2 peripheral is finished.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART 1EN	Res.	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	Reserved		IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.	AFIO EN
	rw			rw	rw			rw	rw	rw	rw	rw	rw		

Figure 10: Registre RCC_APB2ENR

9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 11: Registre GPIOx_CRL

```

    #if BUTTON_ACTIVE_LOW == 1u
        bouton = !bouton; // appuyé = 1
    #endif

    return bouton;
}

```

Explications du code et des configurations de registres utilisées

Cette fonction lit le bit 5 de GPIOB_IDR.

```
bouton = (GPIOB_IDR & BUTTON_MASK) ? 1u : 0u;
```

Le masque garde seulement l'état de PB5. Comme le bouton est actif à 0, la valeur est inversée. La fonction retourne donc 1 quand le bouton est appuyé.

Registres utilisés

Registre	Adresse	Utilité
GPIOB_IDR	0x40010C08	Lit l'état logique de PB5.

Justification avec la documentation officielle

9.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Address offset: 0x08h

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y= 0 .. 15)

These bits are read only and can be accessed in Word mode only. They contain the input value of the corresponding I/O port.

Figure 12: Registre GPIOx_IDR

Explication de la documentation :

La documentation montre que chaque bit de GPIOx_IDR donne l'état de la broche correspondante. Le bit 5 correspond à PB5.

3.8 Fonction SysTick_Init()

Capture du code de la fonction

```
void SysTick_Init(void)
{
    SYST_CSR = 0u; // arrête SysTick
    tempsMs = 0u;

    SYST_RVR = 7999u; // 1 ms avec 8 MHz
    SYST_CVR = 0u; // remet à zéro

    SYST_CSR = CLKSOURCE | TICKINT | ENABLE; // démarre SysTick
}
```

Explications du code et des configurations de registres utilisées

Cette fonction configure SysTick comme base de temps de 1 ms.

8 MHz / 1000 = 8000 cycles

Reload = 8000 - 1 = 7999

SYST_RVR = 7999u;

Cette ligne donne une interruption chaque 8000 cycles.

SYST_CSR = CLKSOURCE | TICKINT | ENABLE;

Cette ligne prend l'horloge du processeur, active l'interruption et démarre SysTick.

Registres utilisés

Registre	Adresse	Utilité
SYST_CSR	0xE000E010	Active SysTick, son interruption et sa source d'horloge.
SYST_RVR	0xE000E014	Contient la valeur de recharge 7999.
SYST_CVR	0xE000E018	Remet le compteur courant à zéro.

Justification avec la documentation officielle

Explication de la documentation :

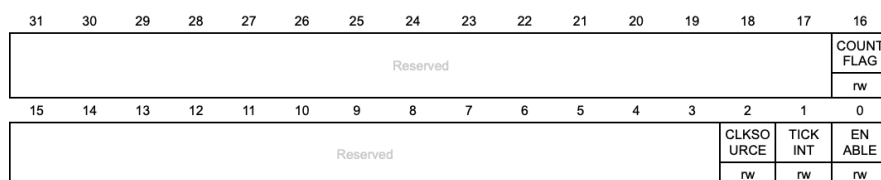
4.5.1 SysTick control and status register (STK_CTRL)

Address offset: 0x00

Reset value: 0x0000 0000

Required privilege: Privileged

The SysTick CTRL register enables the SysTick features.



Bits 31:17 Reserved, must be kept cleared.

Bit 16 **COUNTFLAG**:

Returns 1 if timer counted to 0 since last time this was read.

Bits 15:3 Reserved, must be kept cleared.

Bit 2 **CLKSOURCE**: Clock source selection

Selects the clock source.

0: AHB/8

1: Processor clock (AHB)

Bit 1 **TICKINT**: SysTick exception request enable

0: Counting down to zero does not assert the SysTick exception request

1: Counting down to zero asserts the SysTick exception request.

Note: Software can use COUNTFLAG to determine if SysTick has ever counted to zero.

Bit 0 **ENABLE**: Counter enable

Enables the counter. When ENABLE is set to 1, the counter loads the RELOAD value from the LOAD register and then counts down. On reaching 0, it sets the COUNTFLAG to 1 and optionally asserts the SysTick depending on the value of TICKINT. It then loads the RELOAD value again, and begins counting.

0: Counter disabled

1: Counter enabled

Figure 13: Register SysTick STK_CTRL

4.5.2 SysTick reload value register (STK_LOAD)

Address offset: 0x04

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								RELOAD[23:16]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RELOAD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept cleared.

Bits 23:0 **RELOAD[23:0]**: RELOAD value

The LOAD register specifies the start value to load into the VAL register when the counter is enabled and when it reaches 0.

Calculating the RELOAD value

The RELOAD value can be any value in the range 0x00000001-0x00FFFFFF. A start value of 0 is possible, but has no effect because the SysTick exception request and COUNTFLAG are activated when counting from 1 to 0.

The RELOAD value is calculated according to its use:

- I To generate a multi-shot timer with a period of N processor clock cycles, use a RELOAD value of N-1. For example, if the SysTick interrupt is required every 100 clock pulses, set RELOAD to 99.
- I To deliver a single SysTick interrupt after a delay of N processor clock cycles, use a RELOAD of value N. For example, if a SysTick interrupt is required after 400 clock pulses, set RELOAD to 400.

Figure 14: Register SysTick STK_LOAD

4.5.3 SysTick current value register (STK_VAL)

Address offset: 0x08

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CURRENT[23:16]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CURRENT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept cleared.

Bits 23:0 **CURRENT[23:0]**: Current counter value

The VAL register contains the current value of the SysTick counter.

Reads return the current value of the SysTick counter.

A write of any value clears the field to 0, and also clears the COUNTFLAG bit in the STK_CTRL register to 0.

Figure 15: Register SysTick STK_VAL

Le manuel du Cortex-M3 montre les bits ENABLE, TICKINT et CLKSOURCE dans SYST_CSR. Il montre aussi que la période utilise la valeur de recharge plus 1.

3.9 Fonction SysTick_GetMs()

Capture du code de la fonction

```
unsigned int SysTick_GetMs(void)
{
    return tempsMs;
}
```

Explications du code et des configurations de registres utilisées

Cette fonction retourne le compteur `tempsMs`. Il contient le nombre de millisecondes depuis le démarrage de SysTick.

Registres utilisés

Registre	Adresse	Utilité
Aucun directement	—	La fonction retourne seulement une variable mise à jour par l'interruption SysTick.

Justification avec la documentation officielle

Explication de la documentation :

Cette fonction ne lit pas directement un registre. La configuration de SysTick est expliquée dans `SysTick_Init()`.

3.10 Fonction SysTick_Handler()

Capture du code de la fonction

```
void SysTick_Handler(void)
{
    tempsMs++; // ajoute 1 ms
}
```

Explications du code et des configurations de registres utilisées

Cette fonction est appelée à chaque interruption SysTick. Comme l'interruption arrive chaque 1 ms, elle ajoute 1 au compteur de millisecondes.

Registres utilisés

Registre	Adresse	Utilité
Aucun directement	—	Le drapeau SysTick est géré par le cœur. La fonction augmente seulement <code>tempsMs</code> .

Justification avec la documentation officielle

Explication de la documentation :

La documentation du Cortex-M3 indique que SysTick peut produire une exception quand le compteur arrive à zéro et que TICKINT est activé.

3.11 Fonction Timer2_Stop()

Capture du code de la fonction

```
void Timer2_Stop(void)
{
    NVIC_ICERO = TIM2_IRQ; // coupe IRQ TIM2

    RCC_APB1ENR |= TIM2_EN; // active horloge TIM2

    TIM2_CR1 = 0u; // arrête timer
    TIM2_DIER = 0u; // coupe interruptions
    TIM2_SR = 0u; // efface flags
    TIM2_CNT = 0u; // remet compteur à zéro

    NVIC_ICPRO = TIM2_IRQ; // efface IRQ en attente

    modeTimer = TIMER2_MODE_STOP;
}
```

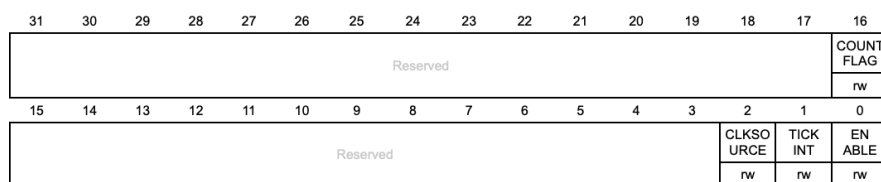
4.5.1 SysTick control and status register (STK_CTRL)

Address offset: 0x00

Reset value: 0x0000 0000

Required privilege: Privileged

The SysTick CTRL register enables the SysTick features.



Bits 31:17 Reserved, must be kept cleared.

Bit 16 **COUNTFLAG**:

Returns 1 if timer counted to 0 since last time this was read.

Bits 15:3 Reserved, must be kept cleared.

Bit 2 **CLKSOURCE**: Clock source selection

Selects the clock source.

0: AHB/8

1: Processor clock (AHB)

Bit 1 **TICKINT**: SysTick exception request enable

0: Counting down to zero does not assert the SysTick exception request

1: Counting down to zero asserts the SysTick exception request.

Note: Software can use COUNTFLAG to determine if SysTick has ever counted to zero.

Bit 0 **ENABLE**: Counter enable

Enables the counter. When ENABLE is set to 1, the counter loads the RELOAD value from the LOAD register and then counts down. On reaching 0, it sets the COUNTFLAG to 1 and optionally asserts the SysTick depending on the value of TICKINT. It then loads the RELOAD value again, and begins counting.

0: Counter disabled

1: Counter enabled

Figure 16: Register SysTick STK_CTRL

Explications du code et des configurations de registres utilisées

Cette fonction arrête TIM2 et enlève les interruptions qui pourraient encore être en attente.

```
RCC_APB1ENR |= TIM2_EN;
```

Le bit TIM2EN active l'horloge de TIM2. Il faut l'horloge pour accéder au timer.

```
TIM2_CR1 = 0u;
TIM2_DIER = 0u;
```

Ces lignes arrêtent le compteur et désactivent son interruption Update.

```
NVIC_ICER0 = TIM2_IRQ;
NVIC_ICPR0 = TIM2_IRQ;
```

Le bit 28 désactive l'IRQ de TIM2 et efface une ancienne demande en attente.

Registres utilisés

Registre	Adresse	Utilité
RCC_APB1ENR	0x4002101C	Active l'horloge de TIM2.
TIM2_CR1	0x40000000	Arrête le compteur.
TIM2_DIER	0x4000000C	Désactive l'interruption Update.
TIM2_SR	0x40000010	Efface les drapeaux de TIM2.
TIM2_CNT	0x40000024	Remet le compteur à zéro.
NVIC_ICER0	0xE000E180	Désactive l'IRQ 28.
NVIC_ICPR0	0xE000E280	Efface l'IRQ 28 en attente.

Justification avec la documentation officielle

Access: word, half-word and byte access

No wait state, except if the access occurs while an access to a peripheral on APB1 domain is on going. In this case, wait states are inserted until this access to APB1 peripheral is finished.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DAC EN	PWR EN	BKP EN	CAN2 EN	CAN1 EN	Reserved		I2C2 EN	I2C1 EN	UART5 N	UART4E N	USART3 EN	USART2 EN	Res.
		rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWD GEN	Reserved					TIM7 EN	TIM6 EN	TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw						rw	rw	rw	rw	rw	rw

Figure 17: Registre RCC_APB1ENR

15.4.1 TIMx control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 18: Registre TIMx_CR1

15.4.5 TIMx status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved			TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0				rc_w0			rc_w0	rc_w0	rc_w0	rc_w0

Figure 19: Registre TIMx_SR

Explication de la documentation :

On voit que le bit CEN de TIM2_CR1 démarre le compteur. Le bit UIF de TIM2_DIER active l'interruption Update et le bit UIF de TIM2_SR indique une mise à jour.

3.12 Fonction Timer2_Start()

Capture du code de la fonction

```
static void Timer2_Start(void)
{
    TIM2_CR1 = URS; // IRQ seulement en fin de période

    TIM2_EGR = UG; // charge PSC et ARR
    TIM2_SR = 0u; // efface UIF
    TIM2_CNT = 0u; // repart de zéro

    NVIC_ICPRO = TIM2_IRQ; // efface ancienne IRQ
    TIM2_DIER = UIE; // active update interrupt
    NVIC_ISERO = TIM2_IRQ; // active IRQ TIM2

    TIM2_CR1 = URS | CEN; // démarre timer
}
```

15.4.10 TIMx counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **CNT[15:0]**: Counter value

15.4.11 TIMx prescaler (TIMx_PSC)

Address offset: 0x28

418/1136

RM0008 Rev 21



RM0008

General-purpose timers (TIM2 to TIM5)

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in "reset mode").

Figure 20: Registres TIMx_CNT et TIMx_PSC

4.3.3 Interrupt clear-enable registers (NVIC_ICERx)

Address offset: 0x00 - 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The ICER0-ICER2 registers disable interrupts, and show which interrupts are enabled.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRENA[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRENA[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:0 **CLRENA[31:0]**: Interrupt clear-enable bits.

Write:

- 0: No effect
- 1: Disable interrupt

Read:

- 0: Interrupt disabled
- 1: Interrupt enabled.

See [Table 41: Mapping of interrupts to the interrupt variables on page 119](#) for the correspondence of interrupts to each register bit.

Figure 21: Registre NVIC_ICER

4.3.5 Interrupt clear-pending registers (NVIC_ICPRx)

Address offset: 0x00 - 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The ICPR0-ICPR2 registers remove the pending state from interrupts, and show which interrupts are pending.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRPEND[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRPEND[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Figure 22: Registre NVIC_ICPR

Explications du code et des configurations de registres utilisées

Cette fonction charge la configuration de TIM2 et démarre le compteur. Elle est `static`, donc elle sert seulement dans `Timer2.c`.

```
TIM2_CR1 = URS;  
TIM2_EGR = UG;
```

URS évite que le UG crée une interruption. UG charge les valeurs de PSC et ARR.

```
TIM2_DIER = UIE;  
NVIC_ISER0 = TIM2_IRQ;
```

Ces lignes activent l'interruption Update dans TIM2 et l'IRQ 28 dans le NVIC.

```
TIM2_CR1 = URS | CEN;
```

Le bit CEN démarre le timer.

Registres utilisés

Registre	Adresse	Utilité
TIM2_CR1	0x40000000	Règle URS et démarre le timer avec CEN.
TIM2_DIER	0x4000000C	Active l'interruption Update.
TIM2_SR	0x40000010	Efface le drapeau UIF.
TIM2_EGR	0x40000014	Produit l'événement UG pour charger PSC et ARR.
TIM2_CNT	0x40000024	Remet le compteur à zéro.
NVIC_ISER0	0xE000E100	Active l'IRQ 28 de TIM2.
NVIC_ICPR0	0xE000E280	Efface une ancienne IRQ en attente.

Justification avec la documentation officielle

Explication de la documentation :

La documentation montre les bits CEN et URS dans `TIM2_CR1`, UIE dans `TIM2_DIER` et UG dans `TIM2_EGR`. TIM2 correspond à l'IRQ 28 dans le NVIC.

15.4.1 TIMx control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 23: Registre TIMx_CR1

15.4.6 TIMx event generation register (TIMx_EGR)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									TG	Res.	CC4G	CC3G	CC2G	CC1G	UG
									w		w	w	w	w	w

Figure 24: Registre TIMx_EGR

15.4.5 TIMx status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved			TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0				rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	

Figure 25: Registre TIMx_SR

15.4.10 TIMx counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **CNT[15:0]**: Counter value

15.4.11 TIMx prescaler (TIMx_PSC)

Address offset: 0x28

418/1136

RM0008 Rev 21



RM0008

General-purpose timers (TIM2 to TIM5)

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in "reset mode").

Figure 26: Registres TIMx_CNT et TIMx_PSC

4.3.2 Interrupt set-enable registers (NVIC_ISERx)

Address offset: 0x00 - 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SETENA[31:16]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETENA[15:0]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **SETENA[31:0]**: Interrupt set-enable bits.

Write:

- 0: No effect
- 1: Enable interrupt

Read:

- 0: Interrupt disabled
- 1: Interrupt enabled.

See [Table 41: Mapping of interrupts to the interrupt variables on page 119](#) for the correspondence of interrupts to each register bit.

If a pending interrupt is enabled, the NVIC activates the interrupt based on its priority. If an interrupt is not enabled, asserting its interrupt signal changes the interrupt state to pending, but the NVIC never activates the interrupt, regardless of its priority.

Figure 27: Registre NVIC_ISER

4.3.5 Interrupt clear-pending registers (NVIC_ICPRx)

Address offset: 0x00 - 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The ICPR0-ICPR2 registers remove the pending state from interrupts, and show which interrupts are pending.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRPEND[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRPEND[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Figure 28: Registre NVIC_ICPR

3.13 Fonction Timer2_PWMInit()

Capture du code de la fonction

```
void Timer2_PWMInit(void)
{
    Timer2_Stop();

    TIM2_PSC = 79u; // tick de 10 us
    TIM2_ARR = 9u; // interruption chaque 100 us

    compteurPWM = 0u;
    modeTimer = TIMER2_MODE_PWM;

    if (PWM_DUTY_PERCENT > 0u)
    {
        LED_Set(LED_ON);
    }
    else
    {
        LED_Set(LED_OFF);
    }

    Timer2_Start(); // démarre PWM
}
```

Explications du code et des configurations de registres utilisées

Cette fonction prépare TIM2 pour le PWM logiciel du mode 1.

```
TIM2_PSC = 79u;
TIM2_ARR = 9u;

PSC = 79
tick = 8 MHz / (79 + 1) = 100 kHz = 10 us
ARR = 9
Update = 10 us × (9 + 1) = 100 us
100 étapes = 10 ms
PWM = 100 Hz
à 69 % : 6,9 ms ON / 3,1 ms OFF
```

Le PWM est fait dans TIM2_IRQHandler(). Ce n'est pas la sortie PWM matérielle de TIM2.

Registres utilisés

Registre	Adresse	Utilité
TIM2_PSC	0x40000028	Divise l'horloge de 8 MHz par 80.
TIM2_ARR	0x4000002C	Produit un Update après 10 ticks.

Justification avec la documentation officielle

15.4.10 TIMx counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **CNT[15:0]**: Counter value

15.4.11 TIMx prescaler (TIMx_PSC)

Address offset: 0x28

418/1136

RM0008 Rev 21



RM0008

General-purpose timers (TIM2 to TIM5)

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.

PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in "reset mode").

Figure 29: Registres TIMx_CNT et TIMx_PSC

Explication de la documentation :

Le registre PSC divise l'horloge par $PSC + 1$. Le compteur va de 0 jusqu'à ARR,

15.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **ARR[15:0]**: Prescaler value
ARR is the value to be loaded in the actual auto-reload register.
Refer to the [Section 15.3.1: Time-base unit](#) for more details about ARR update and behavior.
The counter is blocked while the auto-reload value is null.

Figure 30: Registre TIMx_ARR

donc il compte $ARR + 1$ ticks avant l'Update.

3.14 Fonction Timer2_InterruptInit()

Capture du code de la fonction

```
void Timer2_InterruptInit(void)
{
    Timer2_Stop();

    TIM2_PSC = 7999u; // tick de 1 ms
    TIM2_ARR = 499u;  // interruption chaque 500 ms

    modeTimer = TIMER2_MODE_INTERRUPT;

    Timer2_Start();
}
```

Explications du code et des configurations de registres utilisées

Cette fonction prépare TIM2 pour le mode 2.

```
TIM2_PSC = 7999u;
TIM2_ARR = 499u;

PSC = 7999
tick = 8 MHz / (7999 + 1) = 1 kHz = 1 ms
ARR = 499
```

$\text{Update} = 1 \text{ ms} \times (499 + 1) = 500 \text{ ms}$

TIM2 produit donc une interruption Update chaque 500 ms.

Registres utilisés

Registre	Adresse	Utilité
TIM2_PSC	0x40000028	Divise l'horloge de 8 MHz par 8000.
TIM2_ARR	0x4000002C	Produit un Update après 500 ticks.

Justification avec la documentation officielle

Explication de la documentation :

La documentation montre que PSC règle le diviseur du timer et que l'événement Update arrive quand le compteur atteint ARR.

3.15 Fonction TIM2_IRQHandler()

Capture du code de la fonction

```
void TIM2_IRQHandler(void)
{
    if ((TIM2_SR & UIF) == 0u)
    {
        return;
    }

    TIM2_SR = 0u; // efface flag

    if (modeTimer == TIMER2_MODE_PWM)
    {
        if (compteurPWM < PWM_DUTY_PERCENT)
        {
            LED_Set(LED_ON);
        }
        else
        {
            LED_Set(LED_OFF);
        }

        compteurPWM++;

        if (compteurPWM >= PWM_STEPS)
        {
```

15.4.10 TIMx counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **CNT[15:0]**: Counter value

15.4.11 TIMx prescaler (TIMx_PSC)

Address offset: 0x28

418/1136

RM0008 Rev 21



RM0008

General-purpose timers (TIM2 to TIM5)

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.

PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in "reset mode").

Figure 31: Registres TIMx_CNT et TIMx_PSC

15.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 **ARR[15:0]**: Prescaler value
ARR is the value to be loaded in the actual auto-reload register.
Refer to the [Section 15.3.1: Time-base unit](#) for more details about ARR update and behavior.
The counter is blocked while the auto-reload value is null.

Figure 32: Registre TIMx_ARR

```
        compteurPWM = 0u;
    }
}
else if (modeTimer == TIMER2_MODE_INTERRUPT)
{
    LED_Toggle(); // change LED à chaque interruption
}
}
```

Explications du code et des configurations de registres utilisées

Cette fonction est appelé par l'interruption de TIM2.

```
if ((TIM2_SR & UIF) == 0u)
```

Cette ligne vérifie que l'interruption vient bien d'un événement Update.

En mode PWM, `compteurPWM` va de 0 à 99. La LED est allumée pendant le nombre d'étapes donné par `PWM_DUTY_PERCENT`, puis éteinte pour le reste. Avec 69, elle est ON pendant 69 étapes et OFF pendant 31 étapes.

En mode interruption, `LED_Toggle()` change la LED d'état toutes les 500 ms.

Registres utilisés

Registre	Adresse	Utilité
TIM2_SR	0x40000010	Vérifie et efface le drapeau UIF.

Justification avec la documentation officielle

15.4.5 TIMx status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved			TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0				rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	

Figure 33: Registre TIMx_SR

Explication de la documentation :

Le bit UIF de TIM2_SR passe à 1 quand un événement Update arrive. Le programme l'efface avant de traiter le mode actif.

4. Conclusion

Le programme utilise maintenant PB6 pour la LED et PB5 pour le bouton. SysTick sert de base de temps au mode 0. TIM2 sert au PWM logiciel du mode 1 et aux interruptions du mode 2. Les mesures obtenues correspondent au fonctionnement attendu des trois modes.